

```
"""
```

```
Weekly Canteen Meal Plan Generator
```

```
-----
```

```
Topic      : CSP (Constraint Satisfaction Problem) - Resource Planning
```

```
GUI        : Tkinter
```

```
Level      : Intermediate
```

```
WHAT THIS PROJECT DOES
```

```
-----
```

It assigns one meal to each day of the week (Monday - Sunday) from a canteen menu, using a CSP backtracking algorithm, while satisfying:

1. No meal repeats during the week (AllDifferent constraint)
2. Total weekly cost must stay within budget (Budget constraint)
3. No two consecutive days share the same meal category (Veg/Non-Veg) (Adjacency constraint)
4. Only meals matching the user's chosen diet preference are considered (Domain-filtering constraint)

If no valid plan exists (e.g. budget too low), the program reports that the CSP is unsatisfiable - which is a core CSP concept to mention in your viva/report.

```
"""
```

```
import tkinter as tk
from tkinter import ttk, messagebox
```

```
# -----
```

```
# 1. DATA: The canteen menu (this represents our CSP "domain")
```

```
# -----
```

```
class Meal:
```

```
    def __init__(self, name, category, cost, calories):
        self.name = name
        self.category = category      # "Veg" or "Non-Veg"
        self.cost = cost              # in rupees
        self.calories = calories
```

```
    def __repr__(self):
        return self.name
```

```
MENU = [
    Meal("Veg Thali", "Veg", 60, 650),
```

```

Meal("Paneer Curry + Rice", "Veg", 70, 700),
Meal("Dal Chawal", "Veg", 45, 500),
Meal("Veg Fried Rice", "Veg", 55, 600),
Meal("Chole Bhature", "Veg", 65, 750),
Meal("Nihari", "Non-Veg", 85, 700),
Meal("Veg Sandwich Combo", "Veg", 50, 480),
Meal("Chicken Curry + Rice", "Non-Veg", 90, 800),
Meal("Egg Fried Rice", "Non-Veg", 70, 650),
Meal("Chicken Biryani", "Non-Veg", 100, 900),
Meal("Fish Curry + Rice", "Non-Veg", 95, 750),
Meal("Egg Curry + Roti", "Non-Veg", 65, 600),
Meal("Chicken Sandwich Combo", "Non-Veg", 75, 620),
]

DAYS = ["Monday", "Tuesday", "Wednesday", "Thursday",
        "Friday", "Saturday", "Sunday"]

# -----
# 2. THE CSP SOLVER (Backtracking Search)
# -----

def solve_meal_plan(diet_pref, weekly_budget):
    """
    Returns a dict {day: Meal} if a valid assignment exists,
    otherwise returns None (CSP is unsatisfiable).
    """

    # Filter the domain according to the diet-preference constraint
    if diet_pref == "Any":
        domain = MENU[:]
    else:
        domain = [m for m in MENU if m.category == diet_pref]

    assignment = {}
    used_names = set()

    def backtrack(day_index, remaining_budget):
        # Base case: all 7 days assigned successfully
        if day_index == len(DAYS):
            return True

        day = DAYS[day_index]

        for meal in domain:
            # Constraint 1: AllDifferent (no repeated meal in the week)
            if meal.name in used_names:
                continue

```

```

        # Constraint 2: Budget constraint
        if meal.cost > remaining_budget:
            continue

        # Constraint 3: No same category as previous day
        if day_index > 0:
            prev_meal = assignment[DAYS[day_index - 1]]
            if prev_meal.category == meal.category:
                continue

        # ---- Try this meal (assign) ----
        assignment[day] = meal
        used_names.add(meal.name)

        if backtrack(day_index + 1, remaining_budget - meal.cost):
            return True

        # ---- Undo (backtrack) ----
        del assignment[day]
        used_names.remove(meal.name)

    return False # no meal worked for this day -> trigger backtracking

success = backtrack(0, weekly_budget)
return assignment if success else None

# -----
# 3. GUI (Tkinter)
# -----
class MealPlanApp:
    def __init__(self, root):
        self.root = root
        root.title("Weekly Canteen Meal Plan Generator")
        root.geometry("650x480")
        root.resizable(False, False)

        title = tk.Label(root, text="Weekly Canteen Meal Plan Generator",
                          font=("Arial", 16, "bold"))
        title.pack(pady=10)

        # ---- Input frame ----
        input_frame = tk.Frame(root)
        input_frame.pack(pady=5)

        tk.Label(input_frame, text="Diet Preference:", font=("Arial", 11))\

```

```

        .grid(row=0, column=0, padx=5, pady=5, sticky="e")
self.diet_var = tk.StringVar(value="Any")
diet_menu = ttk.Combobox(input_frame, textvariable=self.diet_var,
                          values=["Any", "Veg", "Non-Veg"],
                          state="readonly", width=15)
diet_menu.grid(row=0, column=1, padx=5, pady=5)

tk.Label(input_frame, text="Weekly Budget (Rs):", font=("Arial", 11))\
    .grid(row=0, column=2, padx=5, pady=5, sticky="e")
self.budget_var = tk.StringVar(value="500")
budget_entry = tk.Entry(input_frame, textvariable=self.budget_var, width=
budget_entry.grid(row=0, column=3, padx=5, pady=5)

generate_btn = tk.Button(root, text="Generate Meal Plan",
                          font=("Arial", 12, "bold"), bg="#4CAF50", fg="w
                          command=self.generate_plan)
generate_btn.pack(pady=10)

# ---- Results table ----
columns = ("Day", "Meal", "Category", "Cost (Rs)", "Calories")
self.tree = ttk.Treeview(root, columns=columns, show="headings", height=8
for col in columns:
    self.tree.heading(col, text=col)
    self.tree.column(col, width=120, anchor="center")
self.tree.pack(pady=10, fill="x", padx=20)

self.total_label = tk.Label(root, text="", font=("Arial", 11, "bold"))
self.total_label.pack(pady=5)

def generate_plan(self):
    # Clear previous results
    for row in self.tree.get_children():
        self.tree.delete(row)
    self.total_label.config(text="")

    # Validate budget input
    try:
        budget = float(self.budget_var.get())
    except ValueError:
        messagebox.showerror("Invalid Input", "Please enter a valid numeric b
        return

    diet_pref = self.diet_var.get()
    plan = solve_meal_plan(diet_pref, budget)

    if plan is None:
        messagebox.showwarning(

```

```

        "No Solution",
        "No valid meal plan found under these constraints.\n"
        "Try increasing the budget or changing diet preference."
    )
    return

total_cost = 0
for day in DAYS:
    meal = plan[day]
    self.tree.insert("", "end", values=(day, meal.name, meal.category,
                                         meal.cost, meal.calories))

    total_cost += meal.cost

self.total_label.config(
    text=f"Total Weekly Cost: Rs {total_cost} | Budget: Rs {budget}"
)

# -----
# 4. RUN THE APPLICATION
# -----
if __name__ == "__main__":
    root = tk.Tk()
    app = MealPlanApp(root)
    root.mainloop()

```